University of Bremen
Department of Computer Science
Prof. Dr. h.c. Michael Beetz, PhD

AICOR Institute for Artificial Intelligence
WiSe 2026/2027
Solution for Worksheet 3

Solution: Exercises for the Lecture Advanced Methods of AI

Asking, Describing, Constraining, Inferring

Tutorial: Thursday, 12.11.2026, 8:30–10:00, room 0.30, ECO 5

Work in pairs with one laptop: open the notebook `03-pizza_queries.ipynb` on Binder (link on Stud.IP). Parts marked **PAPER** are done on this sheet, **NOTEBOOK** in the notebook. Before every **PREDICT**, commit to an answer on this sheet *before* you run the cell.

PROLOGUE

Thursday evening, 19:30

≈ 5 min

> *A small pizzeria in Bremen. A service robot works between the counter and the tables. Seven pizzas are on the counter, three tables have ordered, and two guests must not eat certain things. The robot knows all of this as objects: the classes of `pizza.owl` from last week, plus guests and orders.*

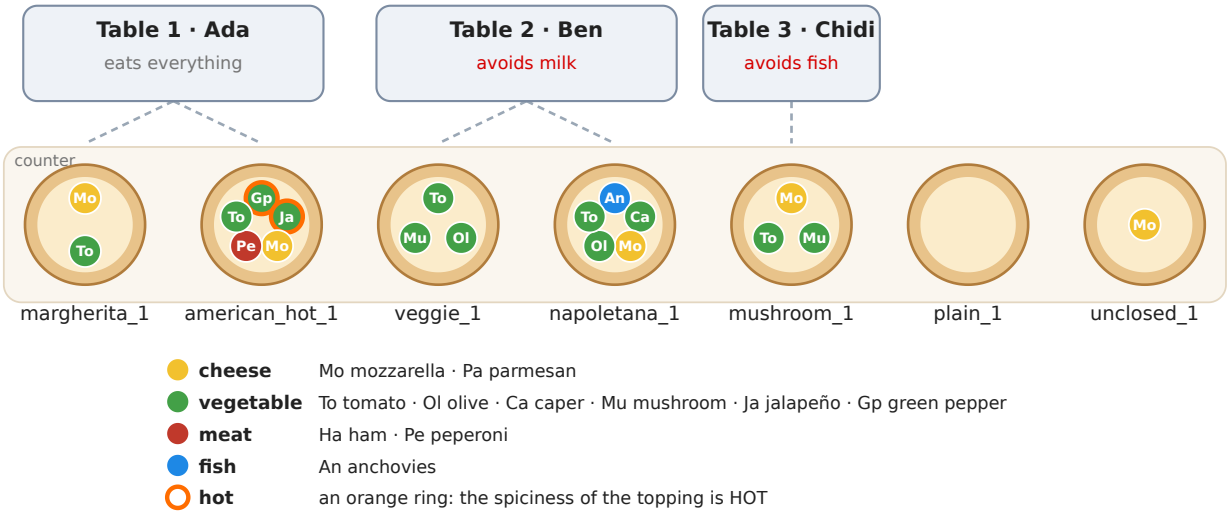


Figure 1. What the robot knows at 19:30. Lines connect each table to the pizzas it ordered.

Everything the robot does tonight starts with the same kind of expression, a **description**. In EQL, the query language of KRROOD, “a pizza with a cheese topping” is written as follows; it is the concept of Worksheet 2 below it:

```
a(Pizza)(toppings=a(CheeseTopping)())
```

$$\texttt{Pizza} \sqcap \exists \texttt{hasTopping.CheeseTopping}$$

What the robot *does* with a description is what this tutorial is about. There are five things, one per act:

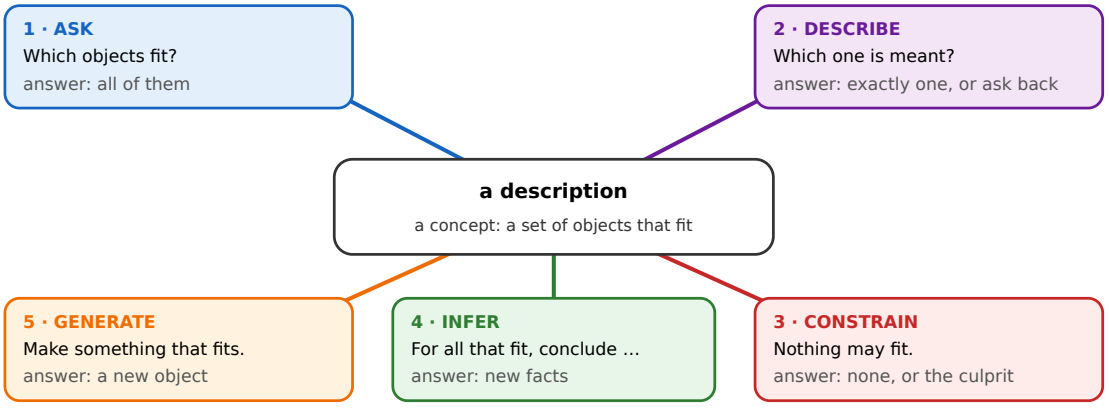


Figure 2. One description, five uses. The expression stays the same; what changes is where the candidates come from and what is done with the objects that fit.

Warm-up

The domain is the set of all objects: 7 pizzas, their 21 toppings, 3 guests, 3 orders (and the values of the enumerations). $\text{CheeseTopping}^{\mathcal{I}}$ is the set of yellow dots (7 mozzarellas); $\text{hasTopping}^{\mathcal{I}}$ is the set of pairs (pizza, dot on that pizza). Every query today is evaluated in this *one* interpretation.

ACT 1 · ASKING

"Which pizzas have cheese?"

≈ 25 min

> *The chef calls across the kitchen. The cheesy pizzas go back into the oven for a minute, so that the cheese melts. The robot has to find them.*

(a) PAPER **A description is a concept.** Write `D = a(Pizza)(toppings=a(CheeseTopping)())` as a first-order formula with one free variable `x`. PEN Ring the pizzas of Figure 1 that are answers. NOTEBOOK Run Act 1(a) and compare.

Solution

`Pizza(x) ∧ ∃y (hasTopping(x, y) ∧ CheeseTopping(y))`. The free variable is what is asked for. Answers: margherita_1, napoletana_1, american_hot_1, mushroom_1, unclosed_1 (Figure 3). A nested description on the list `toppings` means *some* element fits: it is the ∃ of the concept.

The query does what you did by hand in Worksheet 2, Task 2(a): it computes the extension of a concept in one interpretation (model checking). It does not ask whether something is *entailed*, i.e. true in every model.

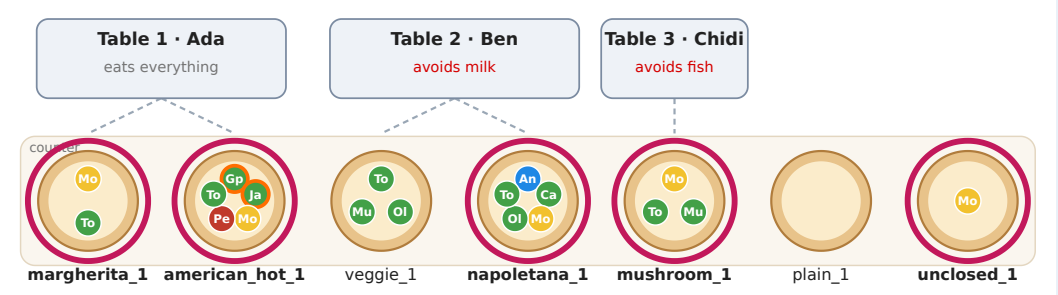


Figure 3. The answers of Act 1(a), ringed.

Three languages, one meaning.

Description logic	First-order logic	EQL
`Pizza`	`Pizza(p)`	`p = variable(Pizza, domain=pizzas)`
a filler of hasTopping	`hasTopping(p, t)`	`t = flat_variable(p.toppings)`
`∃hasTopping.C`	`∃t (hasTopping(p, t) ∧ C(t))`	`exists(t, HasType(t, C))`, or `toppings=a(C)()`
`∀hasTopping.C`	`∀t (hasTopping(p, t) → C(t))`	`for_all(t, HasType(t, C))`
`¬ ⊓ ⊔`	`¬ ∧ ∨`	`not_(…)`, several conditions, `or_(…)`

(b) PREDICT **Some, only, none.** Mark with a cross which pizzas each query returns, and write each query as a DL concept. Then run Act 1(b).

```
Q1 = entity(p).where(for_all(t, HasType(t, VegetableTopping)))
Q2 = a(Pizza)(toppings=[a(MozzarellaTopping)(), a(TomatoTopping)()]).from_(pizzas)
Q3 = entity(p).where(not_(exists(t, HasType(t, CheeseTopping))))
```

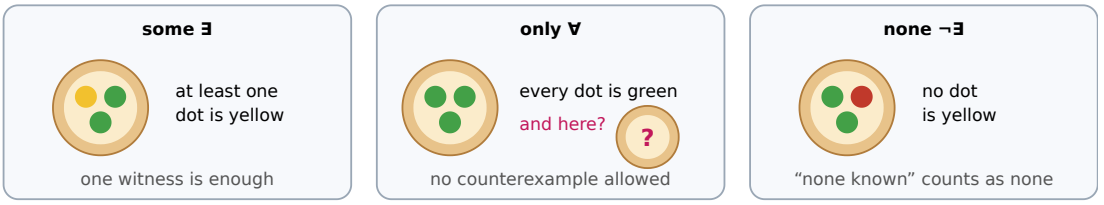


Figure 4. The three quantifier patterns of Worksheet 2, as the robot checks them on one pizza.

Solution

	margherita	napoletana	american_hot	mushroom	veggie	plain	unclosed	DL concept
Q1					×	×		`Pizza ⊓ ∀hasTopping.VegetableTopping`
Q2	×							`Pizza ⊓ ∃hasTopping.Mozzarella ⊓ ∃hasTopping.Tomato ⊓ ∀hasTopping.(Mozzarella ⊔ Tomato)`
Q3					×	×		`Pizza ⊓ ¬∃hasTopping.CheeseTopping`

- **Q1:** plain_1 has no toppings, so “every topping is a vegetable” is vacuously true: there is no counterexample. It is p3 of Worksheet 2.
- **Q2:** a *list* of descriptions describes the whole list: a mozzarella, a tomato, nothing else. That is “some + only”, the closure axiom of the Margherita in Worksheet 2, Task 2(b)iii, in one expression. mushroom_1 has a third topping, unclosed_1 only one. (Strictly, the list also fixes the order and the number, which DL cannot say.)
- **Q3:** nobody stated “veggie_1 has no cheese”. The query concludes it because no cheese is *known*: negation as failure.

? Last week Protégé refused to classify `UnclosedPizza` (a pizza with some mozzarella) as vegetarian. NOTEBOOK Write the query `vegetarian` (no meat, no fish). Is unclosed_1 an answer? Who is right, the query or the reasoner?

Solution

```
vegetarian = entity(p).where(not_(exists(t, or_(HasType(t, MeatTopping), HasType(t,
FishTopping)))))
```

Answers: margherita_1, mushroom_1, veggie_1, plain_1, unclosed_1. In DL: `Pizza ⊓ ¬∃hasTopping.(MeatTopping ⊔ FishTopping)`.

Both are right; they answer different questions. The query treats the objects as *complete*: what is not in the list does not exist (closed world). The reasoner asks whether "vegetarian" holds in *every* model of what was said; there are models in which the pizza also has ham (open world). For a robot looking at a pizza, the closed-world reading is usually what it wants, as long as its knowledge of the pizza is complete.

(c) Following the relations. *Ada at table 1 does not like spicy food: "Is anything hot on my table?"* Write the orders that contain a pizza with a hot topping as a DL concept (roles hasPizza, hasTopping, hasSpiciness; the individual Hot). NOTEBOOK Write `hot_pizzas` and `hot_orders`.

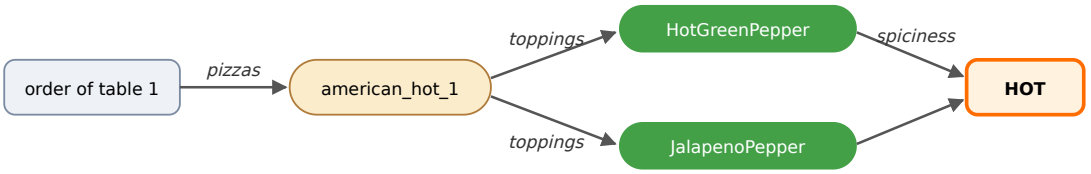


Figure 5. The path from Ada's order to a hot topping. There are two such paths.

PREDICT The notebook then asks for the witnesses themselves: `set_of(o.guest, ordered, topping).where(topping.spiciness == Spiciness.HOT)`. How many answers does it have, while `hot_orders` has one? Answer: _____

Solution

`∃hasPizza.∃hasTopping.∃hasSpiciness.{Hot}`; concepts nest as deep as the objects do.

```
hot_pizzas = a(Pizza)(toppings=a(PizzaTopping)(spiciness=Spiciness.HOT)).from_(pizzas)
hot_orders = an(Order)(pizzas=a(Pizza)(toppings=a(PizzaTopping)
(spiciness=Spiciness.HOT))).from_(orders)
```

Answers: american_hot_1 and the order of table 1. The second query has **two** answers, one per path of Figure 5. An answer is a **binding**, one value per free variable that makes the formula true. In `hot_orders` the pizza and the topping are bound by ∃, so only the order is free and it is returned once, however many witnesses exist.

ACT 2 · DESCRIBING

"Bring me the pizza with olives."

≈ 12 min

> *Commands do not name objects; they describe them. Ben at table 2 waves at the robot.*

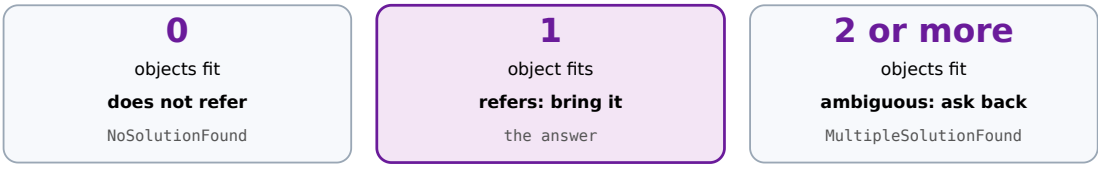


Figure 6. `a(…)` accepts any object that fits. `the(…)` claims that exactly one fits: a definite description.

(a) PREDICT ***a* or *the*?** For each command, how many pizzas fit, and what should the robot do? Then run Act 2(a).

Solution

Command	Pizzas that fit	What the robot does
"the pizza with ham"	none	reports that there is no such pizza (NoSolutionFound)
"the pizza with fish"	napoletana_1	brings it
"the pizza with olives"	napoletana_1, veggie_1	asks back (MultipleSolutionFound)

A definite description presupposes that exactly one object fits (Russell). If the presupposition fails, the command does not refer, and picking one at random would be a guess, not obedience.

(b) Making a description precise. The robot asks back: “The one with fish, or the one without?” Ben: “Without fish.” Write the precise description as a DL concept. NOTEBOOK Write it in EQL so that `the(…)` succeeds.

Solution

`Pizza ⊓ ∃hasTopping.OliveTopping ⊓ ¬∃hasTopping.FishTopping`; its extension tonight is {veggie_1}.

```
olives_without_fish = the(entity(p).where(exists(t, HasType(t, OliveTopping)),
                                          not_(exists(t2, HasType(t2, FishTopping)))))
```

Two quantifiers need two variables over the toppings: one topping is an olive, and *no* topping is a fish. A good question back names a property that one candidate has and the other has not; finding it is again a query over the candidates.

? “The pizza with cheese” does not refer tonight (five fit). Does “the pizza with cheese *of table 3*” refer? What does this tell you about `the`, and about how people resolve “the …” in everyday speech?

Solution

Yes: mushroom_1 is the only cheesy pizza Chidi ordered. Whether a description refers depends on the **domain** it is evaluated in, not only on the description. People shrink the domain with context (the table, what is in sight, what was just said) until “the” works; a robot has to add that context to the description explicitly.

ACT 3 · CONSTRAINING

“Ben must never get milk.”

≈ 15 min

> *Ben avoids milk. Nowhere does it say “napoletana_1 contains milk”: it has to be computed. Mozzarella is a cheese, cheese contains milk, the pizza has that topping.*

(a) PAPER **Stored or computed?** In the notebook, `symbolic_function` turns a Python function into vocabulary of the query language, and a `Predicate` is a relation decided by a computation. For each relation below, is it stored in the objects (S) or computed (C)? NOTEBOOK Complete `allergens` and `IsSafeFor`.

Solution

hasTopping(x, y)	guest(o) = g	MozzarellaTopping ⊏ CheeseTopping	contains(x, milk)	IsSafeFor(x, g)
S: the list `toppings`	S: the field `guest`	S: the class hierarchy	C: `allergens`	C: the predicate

```
@symbolic_function
def allergens(pizza):
    found = {Allergen.GLUTEN}
    if any(isinstance(t, CheeseTopping) for t in pizza.toppings): found.add(Allergen.MILK)
    if any(isinstance(t, FishTopping) for t in pizza.toppings):   found.add(Allergen.FISH)
    return found

class IsSafeFor(Predicate):
    def __call__(self): return not (allergens(self.pizza) & self.guest.avoids)
```

In a query, stored and computed relations look the same. That a symbol's meaning is a program is called procedural attachment. “What can Ben eat?” has the answers veggie_1 and plain_1.

(b) PAPER **A constraint is a query that must have no answer.** The kitchen's rule is: every pizza of an order is safe for the guest of that order. Write it in first-order logic (use hasPizza, guest and IsSafeFor). A query cannot prove a ∀, but it can search for a counterexample: write that formula too.

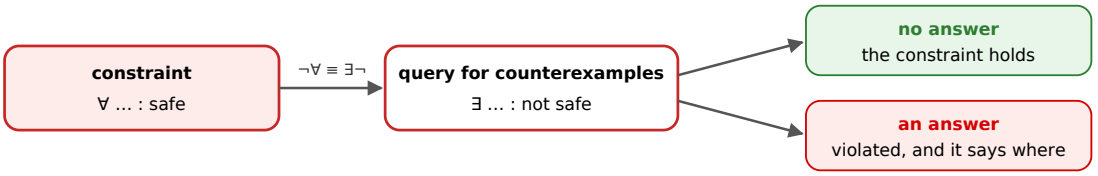


Figure 7. Checking a universal statement over one interpretation means looking for a counterexample.

PREDICT How many violations are there tonight? PEN Draw each one as a red line in Figure 1. NOTEBOOK Write `dangerous` and check.

Solution

Constraint: `∀o ∀x (hasPizza(o, x) → IsSafeFor(x, guest(o)))`.
Counterexample: `∃o ∃x (hasPizza(o, x) ∧ ¬IsSafeFor(x, guest(o)))`, with o and x left free so that the answer names them:

```
dangerous = set_of(o, ordered).where(not_(IsSafeFor(ordered, o.guest)))
```

One violation: table 2, Ben, napoletana_1 (mozzarella, hence milk). Nobody wants to read this list; the kitchen wants it to be empty. But when it is not, the binding is exactly the useful part: the robot knows which plate not to serve.

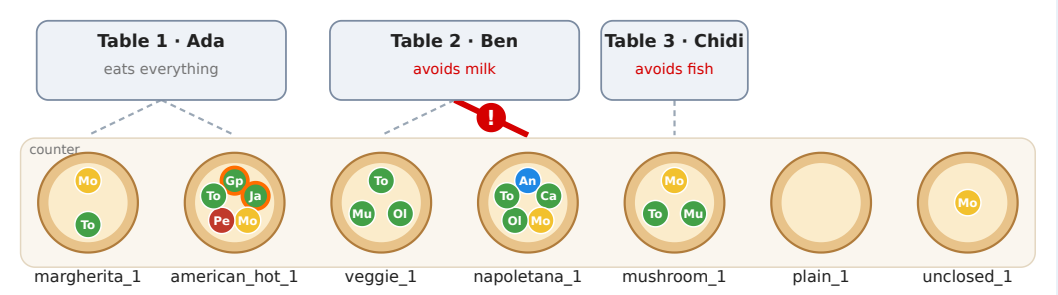


Figure 8. The one violation of the constraint.

? In OWL you could state the kitchen's rule as an axiom. If the facts violate it, what does the reasoner report? What does the query report instead?

Solution

In OWL an axiom does not check the facts, it restricts the models. Facts that violate it leave no model: the ontology is **inconsistent**, everything is entailed, and the reasoner says "inconsistent" (its explanation lists the axioms involved). If a fact were merely missing, e.g. the mozzarella of the Napoletana, the open-world reasoner would find no violation at all. The query treats the rule as an *integrity constraint* over one interpretation: the knowledge base stays usable, and the answer is the culprit.

ACT 4 · INFERRING ≈ 20 min

The kitchen writes warnings and labels.

> *The menu must list the allergens of every pizza and a label: vegetarian, vegan or meaty. Instead of computing these anew for every question, the kitchen writes them down once, as new facts.*

(a) A rule creates facts. In EQL, `inference(AllergenWarning)(pizza=D, …)` creates one new warning object for every pizza that fits the description D. The milk rule is

```
∀x (Pizza(x) ∧ ∃y (hasTopping(x, y) ∧ CheeseTopping(y)) → ∃w (Warning(w) ∧ about(w, x) ∧ allergen(w, milk)))
```

PAPER Which part of the formula is the description D, and which part does the rule build? PREDICT With a fish rule and a gluten rule (every pizza) added, how many warnings are there? _____ NOTEBOOK Write both rules.

Solution

The left side of → is the body of the rule: exactly the description of Act 1(a). The right side is the head: what is built for each answer.

```
fishy = a(Pizza)(toppings=a(FishTopping)()).from_(pizzas)
fish_warnings   = inference(AllergenWarning)(pizza=fishy, allergen=Allergen.FISH).tolist()
gluten_warnings = inference(AllergenWarning)(pizza=p, allergen=Allergen.GLUTEN).tolist()
```

13 warnings: 5 milk, 1 fish, 7 gluten. Producing all conclusions in advance is **forward chaining** (materialisation), as HermiT did when it filled the inferred class hierarchy. The predicate of Act 3 is the opposite: computed when asked (backward chaining). Both found Ben's problem.

(b) PREDICT **Rules with exceptions.** The chef's rule of thumb for the labels is a rule tree (Figure 9). Predict the label of each pizza, then run Act 4(b).

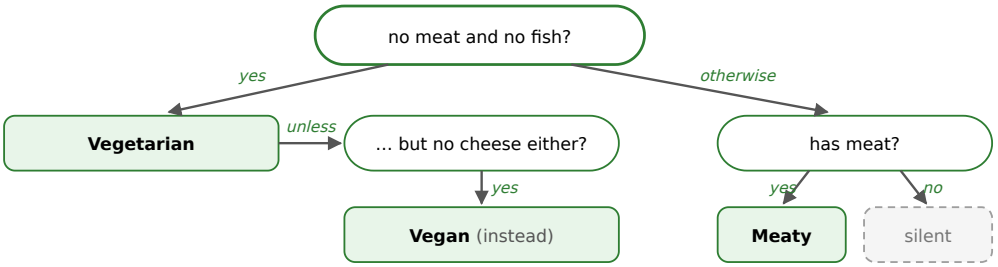


Figure 9. A rule tree: a default conclusion, an exception (*unless*, EQL: `refinement`) and an alternative (*otherwise*, EQL: `alternative`).

Solution

margherita	napoletana	american_hot	mushroom	veggie	plain	unclosed
Vegetarian	**none**	Meaty	Vegetarian	Vegan	Vegan	Vegetarian

napoletana_1 has fish: the first condition fails, and it has no meat, so the alternative fails as well. The rules are **silent**. Silence is not "no": they never concluded "not vegetarian"; reading it so is one more closed-world step. The *unless* branch is an exception: it replaces the general conclusion where a more specific one applies. Trees of this shape are known as ripple-down rules.

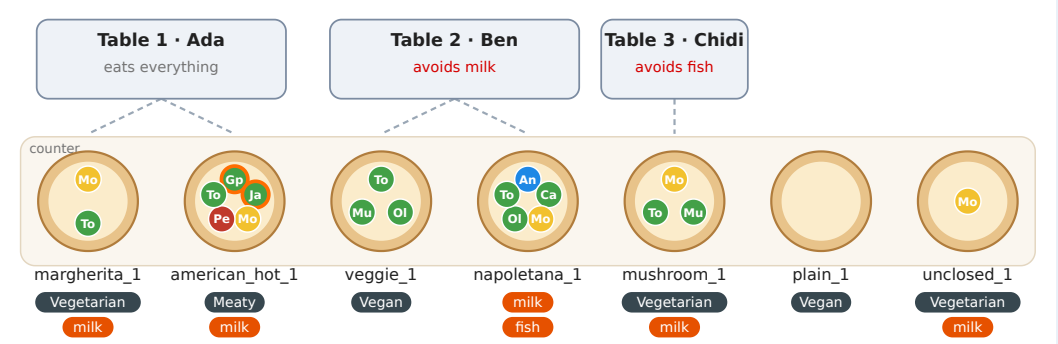


Figure 10. The derived labels and the milk and fish warnings (gluten, on every pizza, is left out).

(c) PREDICT **The world moves on.** *It is busy. The cook mixes up two tickets and grates parmesan over veggie_1, Ben's pizza.* What do the three sources say about veggie_1 now? Which of them protects Ben? Then run Act 4(c).

Solution

1. the constraint of Act 3, checked now	2. the milk warnings derived in (a)	3. the label of veggie_1, derived again
two violations: napoletana_1 *and* veggie_1	no warning for veggie_1	Vegetarian (it was Vegan)

- Only the constraint protects Ben. It was written before the mistake and is evaluated against the world as it is *now*.

- The warnings were concluded at a fixed moment and are stale until the rule runs again. *When* to re-derive is the hard part of forward chaining; a robot, whose world changes all the time, faces it constantly. Keeping derived facts in line with their reasons is called truth maintenance.
- "Vegan" was withdrawn because a fact was *added*. Reasoning in which more knowledge can remove a conclusion is **non-monotonic**. Classical logic and OWL are monotonic: what is entailed stays entailed when axioms are added. Under the open-world assumption OWL would not have concluded "vegan" in the first place.

ACT 5 · GENERATING ≈ 13 min

"A pizza with mozzarella and tomato, please."

> *Dana sits down at table 4 and orders. Her order is a description, exactly like those of Act 1. What is new is who answers it.*

A query in EQL is a symbolic object; nothing is computed when it is written. A **backend** is whatever answers it. The same description can be handed to different backends:

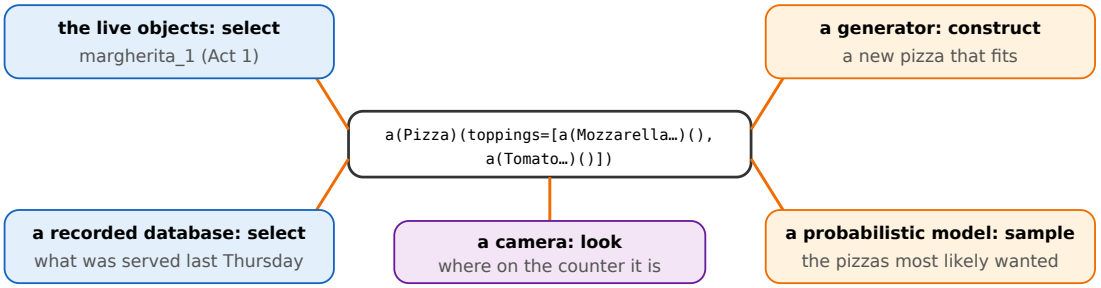


Figure 11. The description is written once, symbolically. Selecting, constructing, sampling and looking are different ways of answering it.

(a) PREDICT **Select or construct?** What does the counter answer, and what does the kitchen answer? Then Dana adds "… and a green pepper, I don't mind how hot" (`spiciness=...`, left open). How many pizzas fit that description? _____ NOTEBOOK Run Act 5.

> ? In Act 1, every answer was an object of the interpretation in Figure 1. What is an answer here? What does it mean, in the language of models, that Dana's open description has several answers?

Solution

The counter (selecting backend) answers margherita_1; the kitchen (generative backend) answers a *new* pizza with exactly these toppings. The open description has three answers, one per spiciness of the pepper.

An answer is now a new object that, added to the interpretation, makes the description true: answering means *constructing a model* instead of checking one. An underspecified description has several models; the generator enumerates them, a probabilistic backend would rank them by how likely they are. Everything Dana left open, the kitchen fills in, just as a cook does.

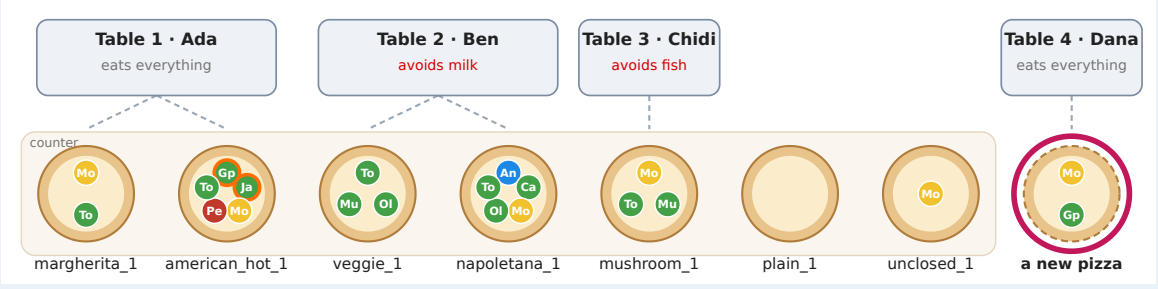


Figure 12. The world has grown: a pizza that did not exist at 19:30.

(b) PAPER **One plan, many answerers.** The robot's next task is a plan with open slots, each written as a description:

Bring ① a pizza that is safe for Ben, lying ② at some pose on the counter, **to** ③ the table of Ben, **holding the plate with** ④ a grasp that works. **If none is on the counter:** ⑤ have one made.

Which kind of backend should answer each slot? Choose from **K** a query over the knowledge base, **R** rules and predicates, **L** looking with the camera, **S** sampling from a probabilistic model, **G** a generator.

Solution

① safe pizza	② its pose	③ Ben's table	④ grasp	⑤ have one made
R (and K): IsSafeFor over the pizzas	L: the knowledge base knows *that* it is there, not exactly *where*	K: the orders	S: sampled, then tried	G: Act 5(a)

Several answers are defensible; what matters is that the plan does not say *who* answers. It states what it wants, symbolically, and each slot is handed to whichever backend can answer that kind of description. This is how the robot of our current research works: in a sorting task, the piece to pick up is *looked for* by perception, where it stands is *read off* the robot's digital twin, the hole it fits through is *concluded* by rules, and how to take hold of it is *sampled*; one plan, four backends. Separating the symbolic description from the machinery that answers it is what lets one language serve questions, constraints, rules, plans and perception.

EPILOGUE ≈ 5 min

The whole picture

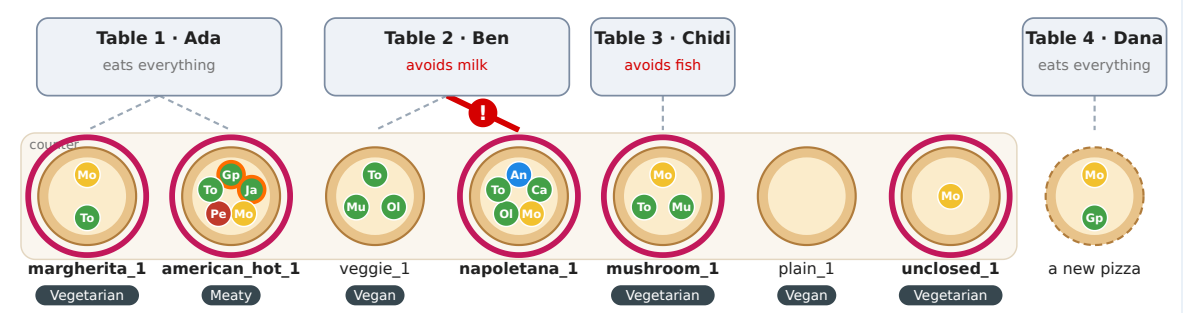


Figure 13. What the robot knows at 20:00: the answers to the chef's question (ringed), the violated constraint (red), the derived labels, and Dana's new pizza.

Take home. One expression, a description, carried the whole evening. Its meaning is a concept, as in description logic; what you *do* with it decides whether it is a question, a reference, a constraint, the body of a rule or a specification. Next week: the same descriptions as the open parameters of robot plans.

(c) PAPER **Your summary.** For each use of a description, give tonight's example and what it looks like in logic.

Solution

Use	The question behind it	Tonight's example	In logic
1 Ask	Which objects fit?	the cheesy pizzas; who gets which hot topping	the extension of a concept, the satisfying assignments of a formula with free variables, in one interpretation
2 Describe	Which one is meant?	the pizza with olives and without fish	a definite description: the extension must have exactly one element
3 Constrain	Does anything violate it?	no pizza unsafe for its guest	a ∀ checked by searching for a counterexample ∃¬; an integrity constraint, not an axiom
4 Infer	What follows for each fit?	allergen warnings, menu labels with an exception	a rule body → head; forward chaining; with exceptions non-monotonic
5 Generate	What could fit?	Dana's pizza; the slots of the plan	model construction: new objects that make the description true